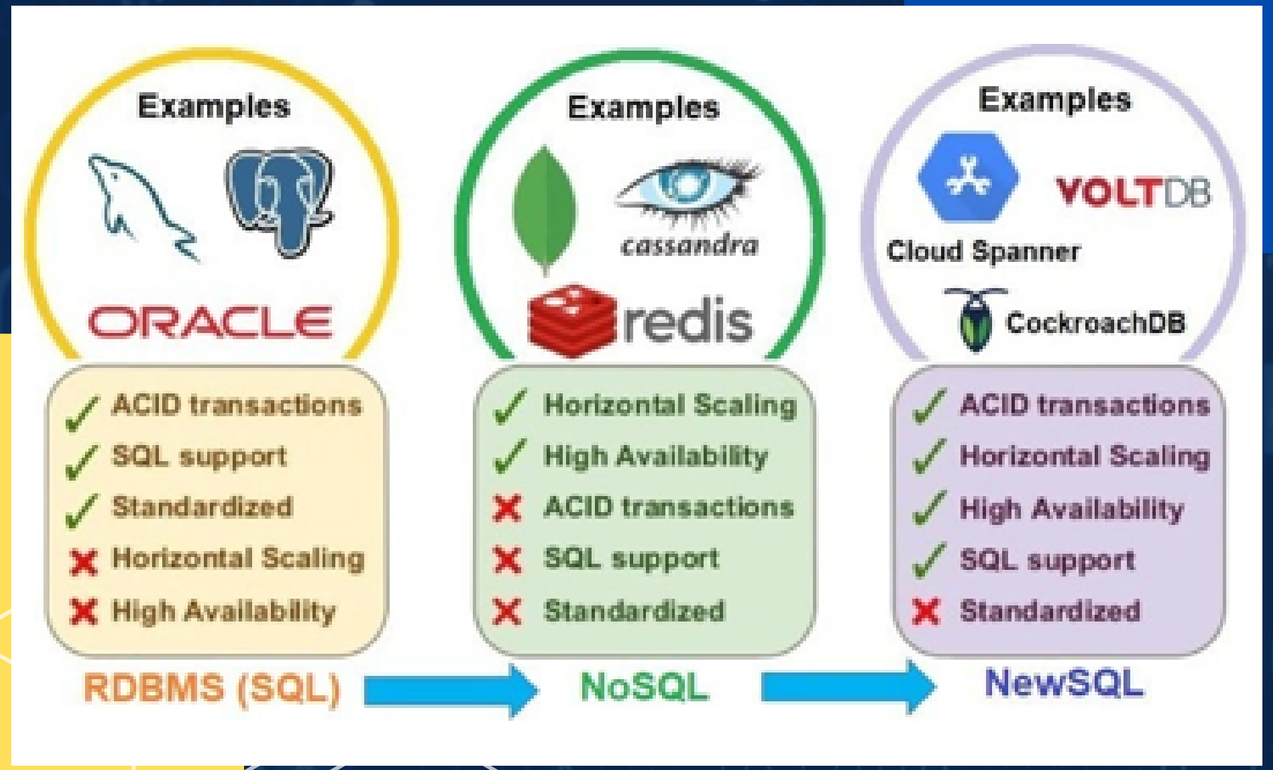
# NEWSQL คืออะไร?

WOOTTIKARN HONGWIENGCHAN



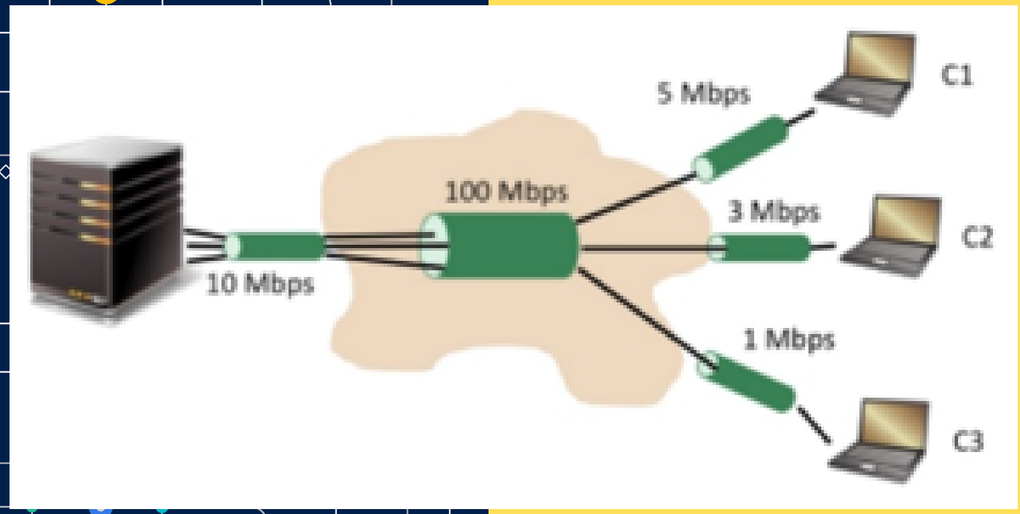

# วิวัฒนาการณ์ของฐานข้อมูล

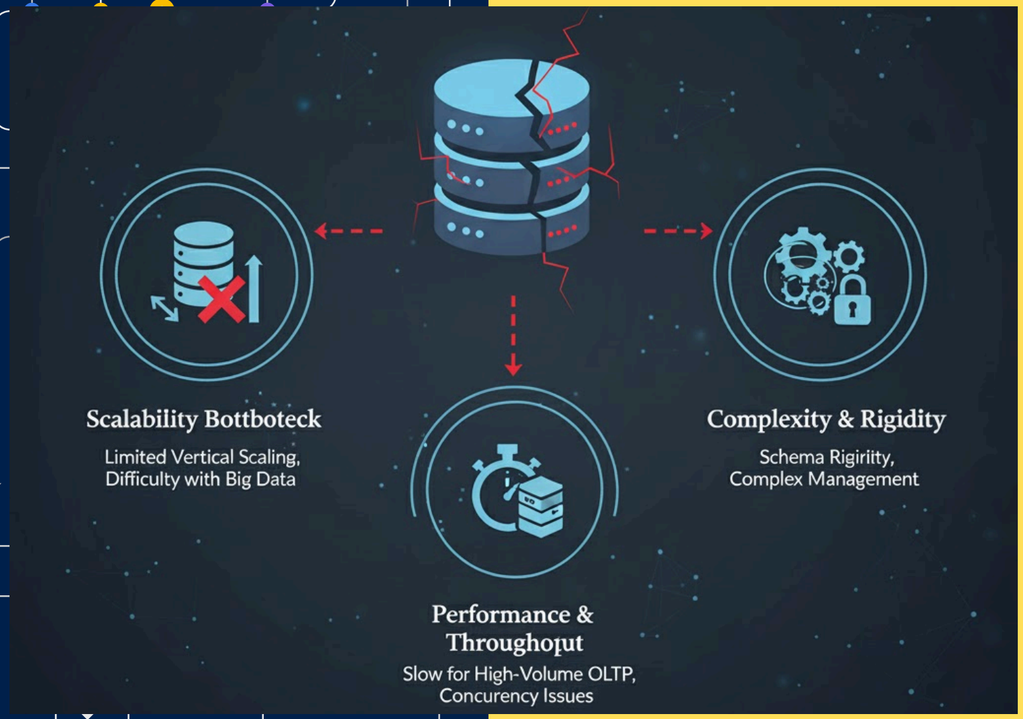
- จาก Relational Database → NoSQL → NewSQL
- NewSQL ถือกำเนิดขึ้นเพื่อตอบโจทย์ทั้งความเร็วและความถูกต้องของข้อมูล
- ใช้ SQL เหมือนเดิมแต่รองรับระบบขนาดใหญ่และ real-time

# ประวัติของ NEWSQL

- คำว่า “NewSQL” ถูกเสนอครั้งแรกในปี 2011 โดย Matthew Aslett
- จุดประสงค์: เพื่ออธิบายฐานข้อมูลรุ่นใหม่ที่คงไว้ซึ่งคุณสมบัติ ACID ของ SQL
- แต่สามารถขยายระบบแบบกระจายข้อมูล (Distributed System) ได้เหมือน NoSQL
- ตัวอย่างยุคแรก เช่น VoltDB, NuoDB, Clustrix
- พัฒนาเพื่อรองรับงาน real-time transaction และ Big Data

# ปัญหาของฐานข้อมูล SQL ดั้งเดิม




- การขยายระบบแนวตั้ง (Vertical Scaling) ทำได้ยาก
- ไม่เหมาะกับข้อมูลขนาดใหญ่แบบ Big Data
- การประมวลผลแบบกระจายทำได้จำกัด
- มี bottleneck ที่ single server หมายถึง ระบบฐานข้อมูลหรือระบบประมวลผลติดขัดหรือชะลอตัว เพราะทุกอย่างต้องผ่านเครื่องเดียว (server เดียว) ซึ่งกลายเป็น “คอขวด” ของระบบ

# ทำไม NoSQL จึงเกิดขึ้น

รองรับข้อมูลไม่เป็นโครงสร้าง เช่น JSON, Document, Graph

- Scale-out ง่าย ขยายระบบได้โดยเพิ่มเครื่อง
- แต่ไม่มีการรับประกัน ACID
- ความถูกต้องของข้อมูล (Consistency) ต่ำกว่า SQL

# จุดกำเนิดของ NEWSQL

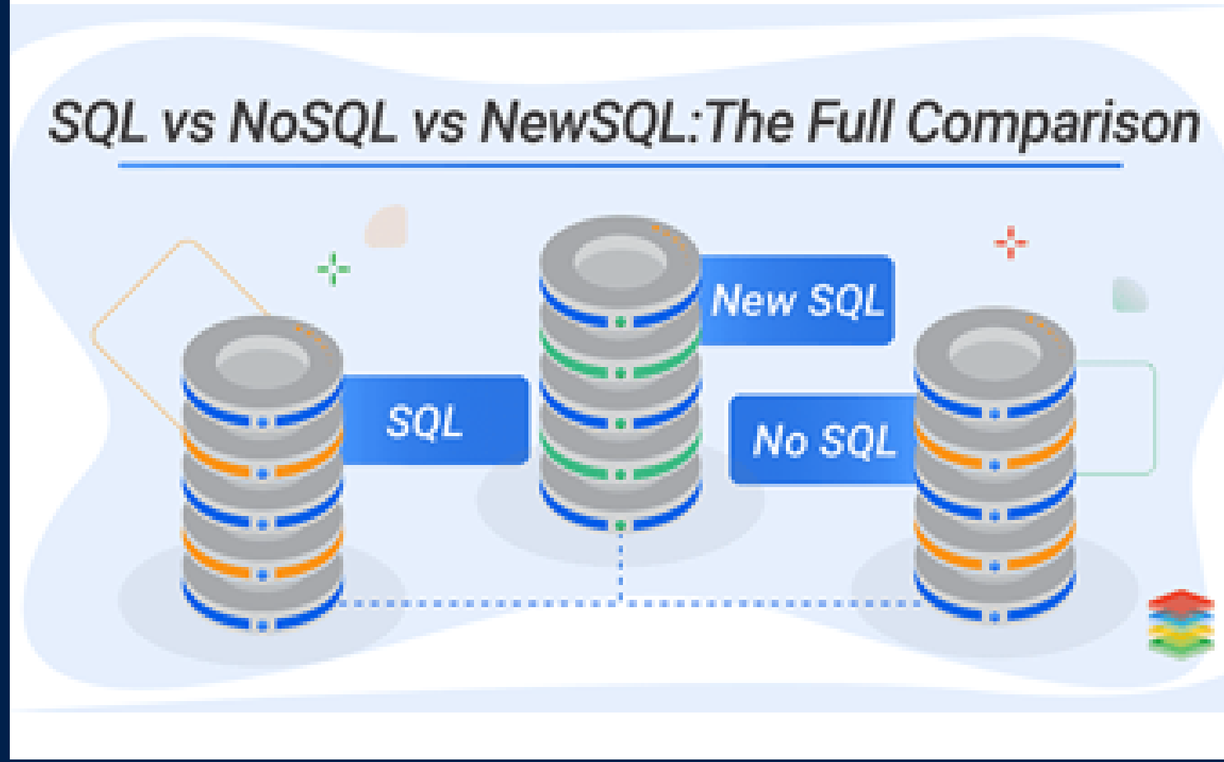
• • เกิดจากความต้องการระบบที่มีทั้งความเร็วของ NoSQL และความถูกต้องของ SQL

• ปี 2011-2012: เริ่มมีฐานข้อมูล NewSQL ตัวแรกออกมา

• จุดเด่นคือ รองรับการทำงานแบบ distributed แต่ยังใช้ SQL เหมือนเดิม

# นิยามของ NEWSQL

- ระบบฐานข้อมูลที่รักษาคุณสมบัติ ACID ได้
- สามารถขยายระบบแบบกระจาย (Horizontal Scaling)
- ใช้ SQL เป็นภาษาหลัก
- มีประสิทธิภาพสูงกว่า RDBMS แบบเดิม



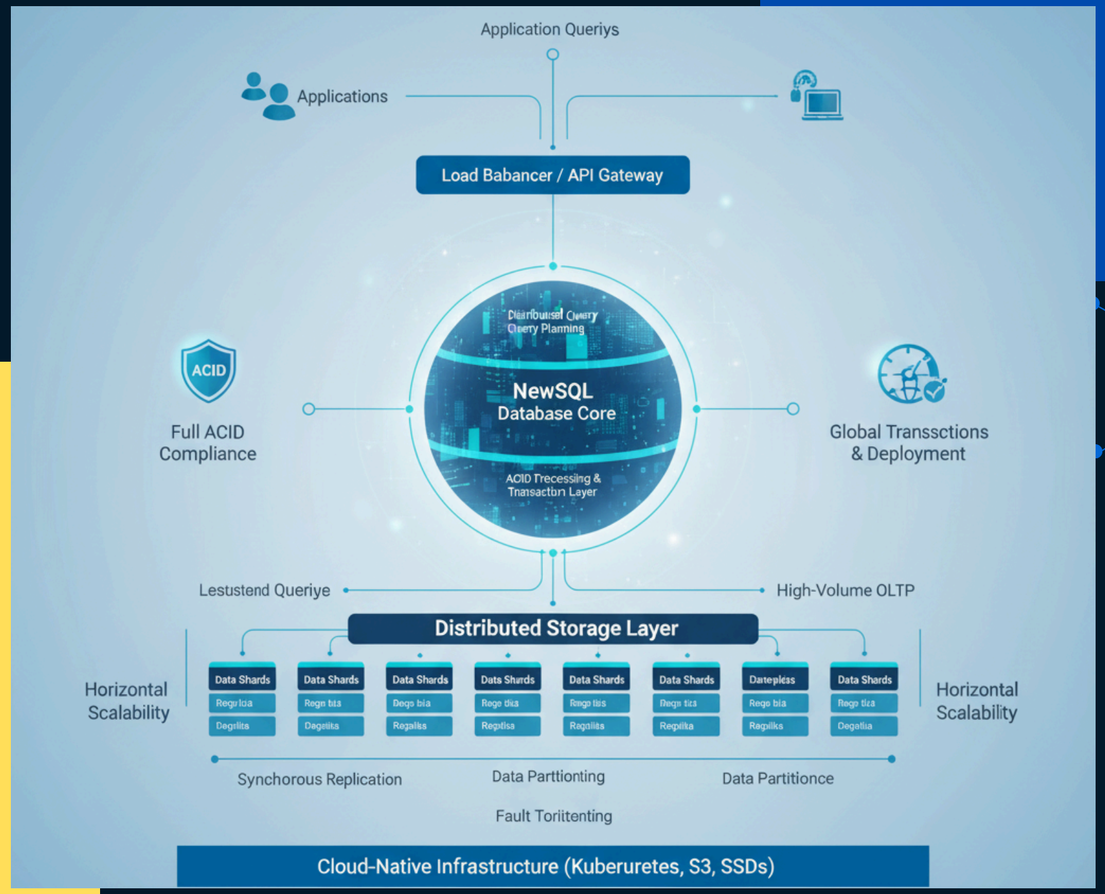

# เปรียบเทียบ SQL / NOSQL / NEWSQL

- SQL: ACID สูง, scale ยาก
- NoSQL: scale ง่าย, consistency ต่ำ
- NewSQL: รวมข้อดีของทั้งสองแบบ

VOLTDB

memSQL

Clus




- In-memory processing
- Distributed transaction management
- Consensus protocols (Paxos, Raft)
- Sharding / Partitioning

# สถาปัตยกรรมของ NEWSQL

ทั้ง 4 ข้อนี้เป็น แนวคิดสำคัญของระบบฐานข้อมูลยุคใหม่ (โดยเฉพาะพวก NewSQL / Distributed Database) แต่ละอย่างมีหน้าที่ช่วยให้ระบบ เร็วขึ้น, ขยายได้ (scalable), และเชื่อถือได้ (reliable)

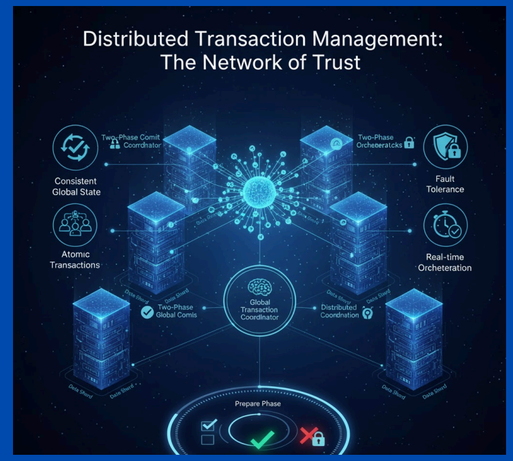
# IN-MEMORY  PROCESSING

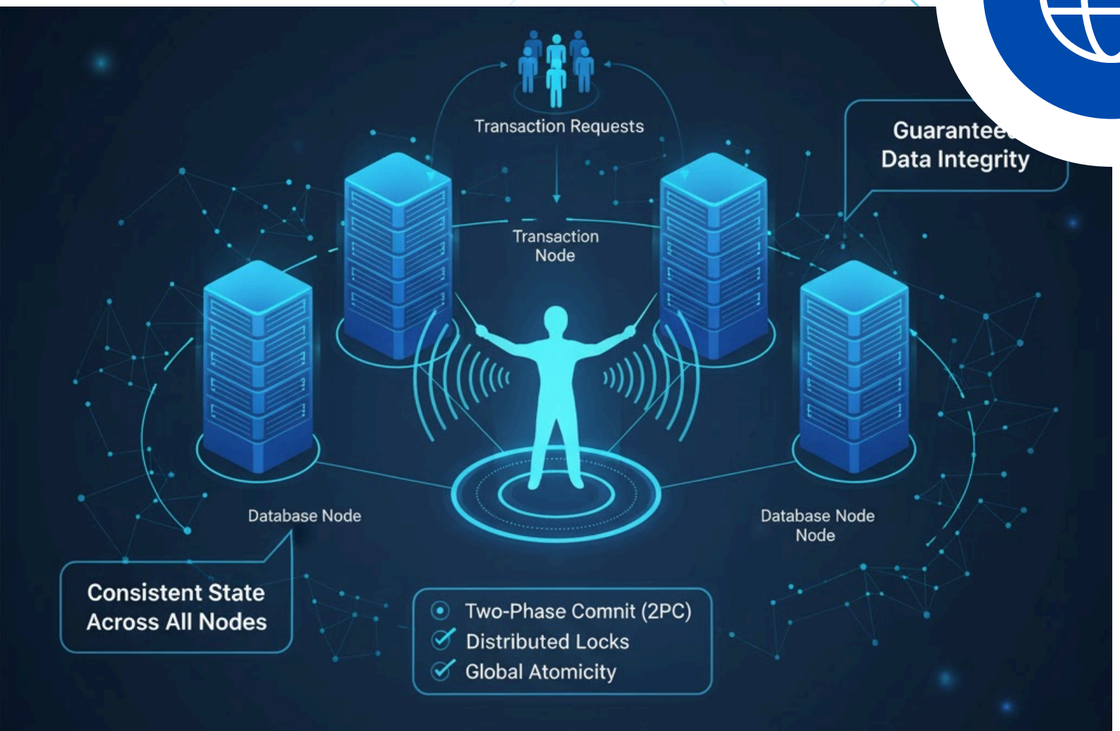
คือการเก็บและประมวลผลข้อมูลใน RAM แทนที่จะดึงจาก Disk

• ปกติฐานข้อมูลแบบดั้งเดิม (SQL เดิม ๆ) จะเก็บข้อมูลบน Hard Disk / SSD → การอ่าน–เขียนข้อมูลต้องใช้เวลา (I/O ช้า)

• แต่ In-Memory Database จะโหลดข้อมูลไว้ใน RAM ซึ่งเร็วกว่าหลายพันเท่า→ ทำให้การ query และประมวลผล แทบจะทันที

# DISTRIBUTED TRANSACTION MANAGEMENT




**คือการจัดการ “ธุรกรรม” ที่เกี่ยวข้องกับ หลายเครื่อง (หลาย node) ให้เหมือนทำงานในเครื่องเดียว**

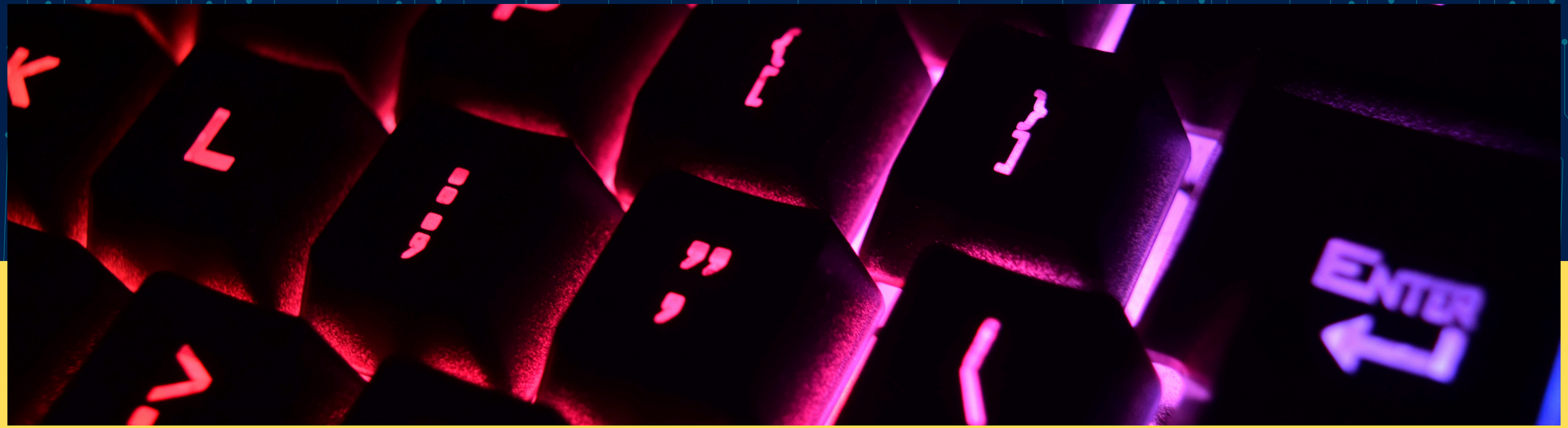
- ในระบบ Distributed Database ข้อมูลไม่ได้อยู่ในเครื่องเดียว แต่กระจายอยู่หลายเครื่อง
- เมื่อมีธุรกรรม (Transaction) ที่ต้องแตะข้อมูลจากหลาย node เช่น การทำธุรกรรมจากบัญชีที่อยู่คนละเครื่อง

→ ระบบต้องจัดการให้ ทุก node ดำเนินการสำเร็จพร้อมกัน หรือยกเลิกพร้อมกัน

# CONSENSUS PROTOCOLS (PAXOS, RAFT)

**คือ “กลไกตกลงร่วมกัน” ระหว่างหลายเครื่องในระบบกระจาย เพื่อให้ได้ “คำตอบเดียวกัน” แม้บางเครื่องจะล่มหรือสื่อสารล่าช้า**

- ในระบบกระจาย (Distributed System) แต่ละเครื่องอาจเห็นข้อมูลไม่ตรงกัน
- ต้องมีกลไกที่ช่วยให้ “ทุกเครื่องเห็นพ้อง” ว่าข้อมูลล่าสุดคืออะไร, ใครคือ leader, อะไรคือสถานะที่ถูกต้อง

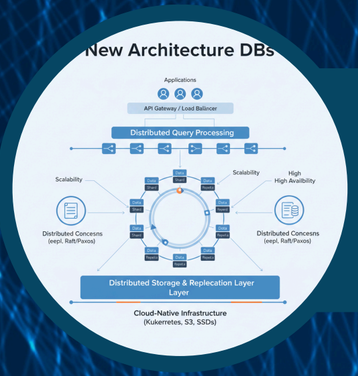


# Sharding / Partitioning

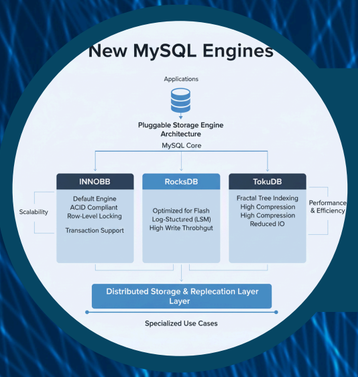
คือการแบ่งข้อมูล “ออกเป็นหลายส่วน (shards)” แล้วกระจายเก็บไว้หลายเครื่องเพื่อเพิ่มความเร็วและรองรับข้อมูลจำนวนมหาศาลe.

- ถ้าฐานข้อมูลโตขึ้นเรื่อย ๆ เครื่องเดียวจะไม่พอ
- จึง “แบ่งข้อมูลออกเป็นส่วน ๆ” แล้วกระจายไปยังเครื่องหลายเครื่อง

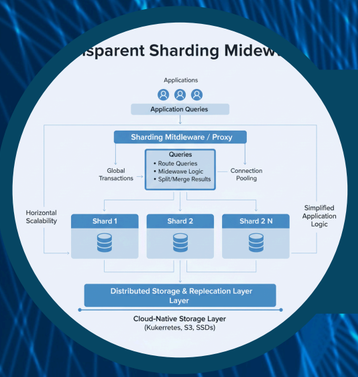
# กลุ่มของ NEWSQL




## New Architecture DBs




## New MySQL Engines



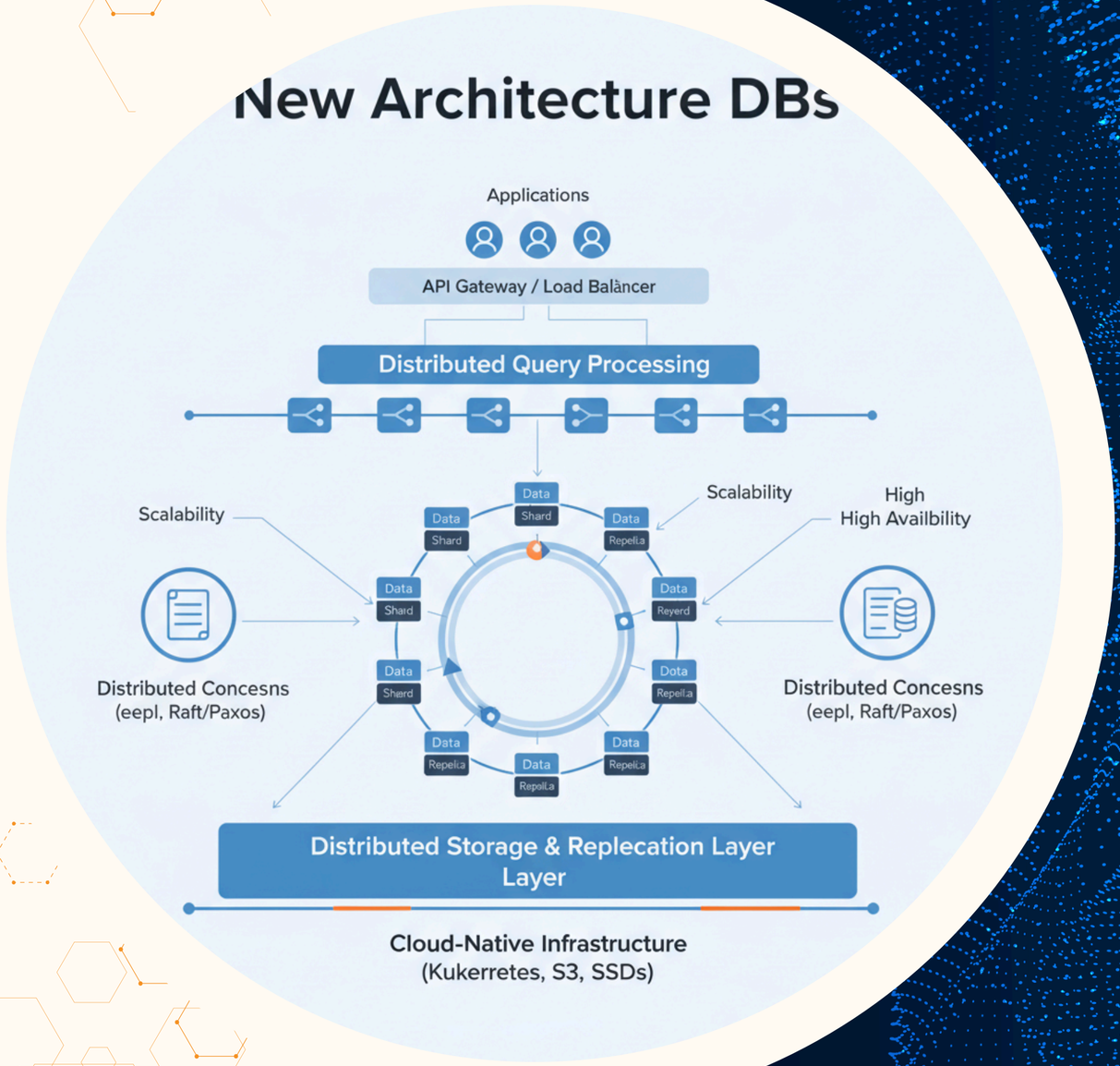

## Transparent Sharding Middleware

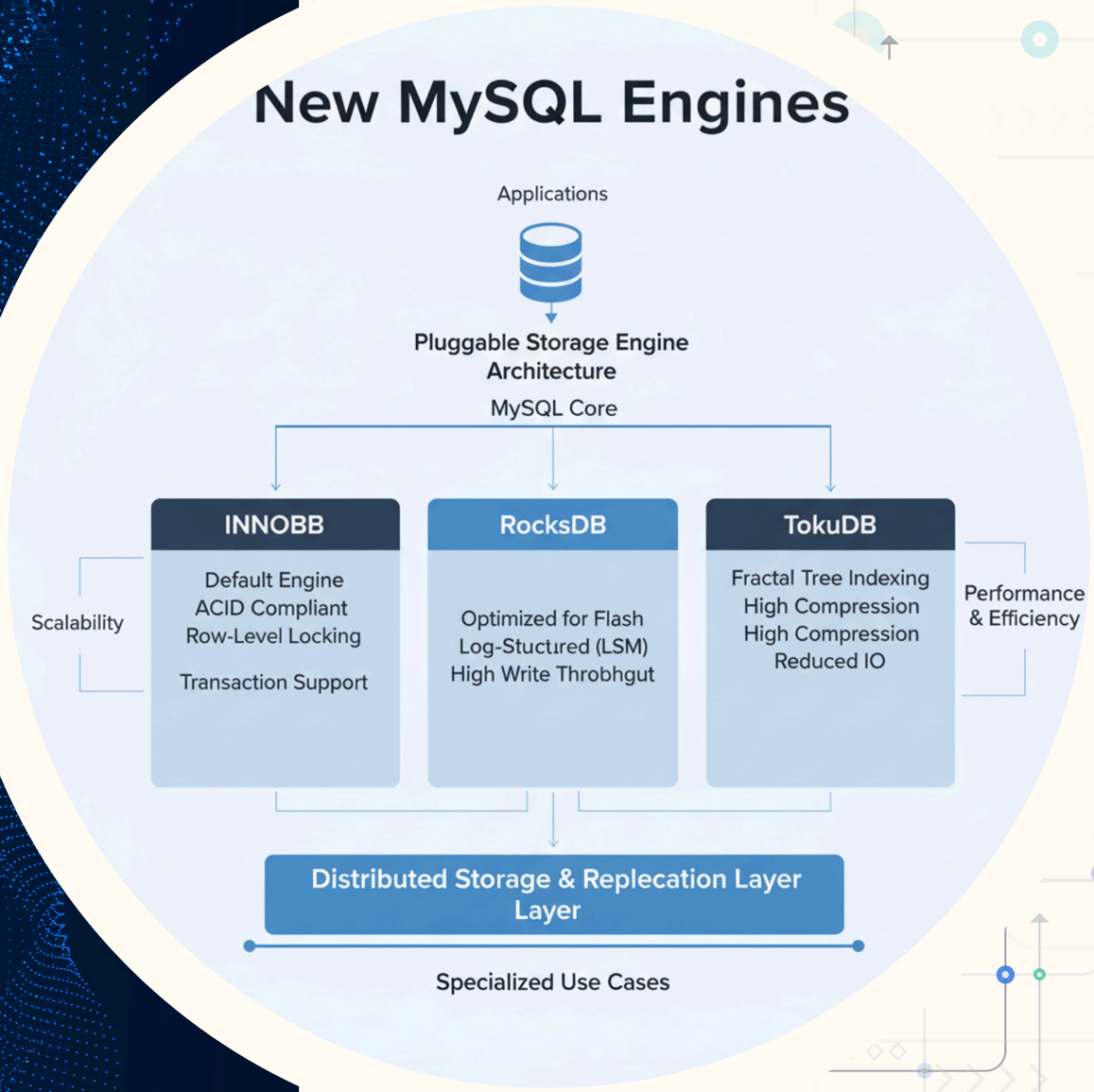
ทั้ง 3 หัวข้อนี้เป็น แนวทางหลักของ NewSQL ซึ่งเกิดขึ้นเพื่อแก้ข้อจำกัดของฐานข้อมูล SQL แบบดั้งเดิม (ที่ช้า / ขยายระบบยาก) แต่ยังคง ภาษาการใช้งาน SQL และคุณสมบัติ ACID ไว้ครบถ้วน)

# NEW ARCHITECTURE DBS

คือ "ฐานข้อมูลที่ออกแบบสถาปัตยกรรมใหม่ทั้งหมด"เพื่อรองรับการกระจายข้อมูล (Distributed System) และการทำงานบน Cloud ได้โดยตรง

- ไม่ได้ใช้โครงสร้างของ MySQL หรือ PostgreSQL เดิมเลย
- ถูกออกแบบใหม่ตั้งแต่ระดับ Storage Engine, Transaction Manager, Consensus Protocol
- ใช้เทคโนโลยีอย่าง Paxos / Raft, TrueTime, Distributed Transaction, Sharding






# NEW MYSQL ENGINES

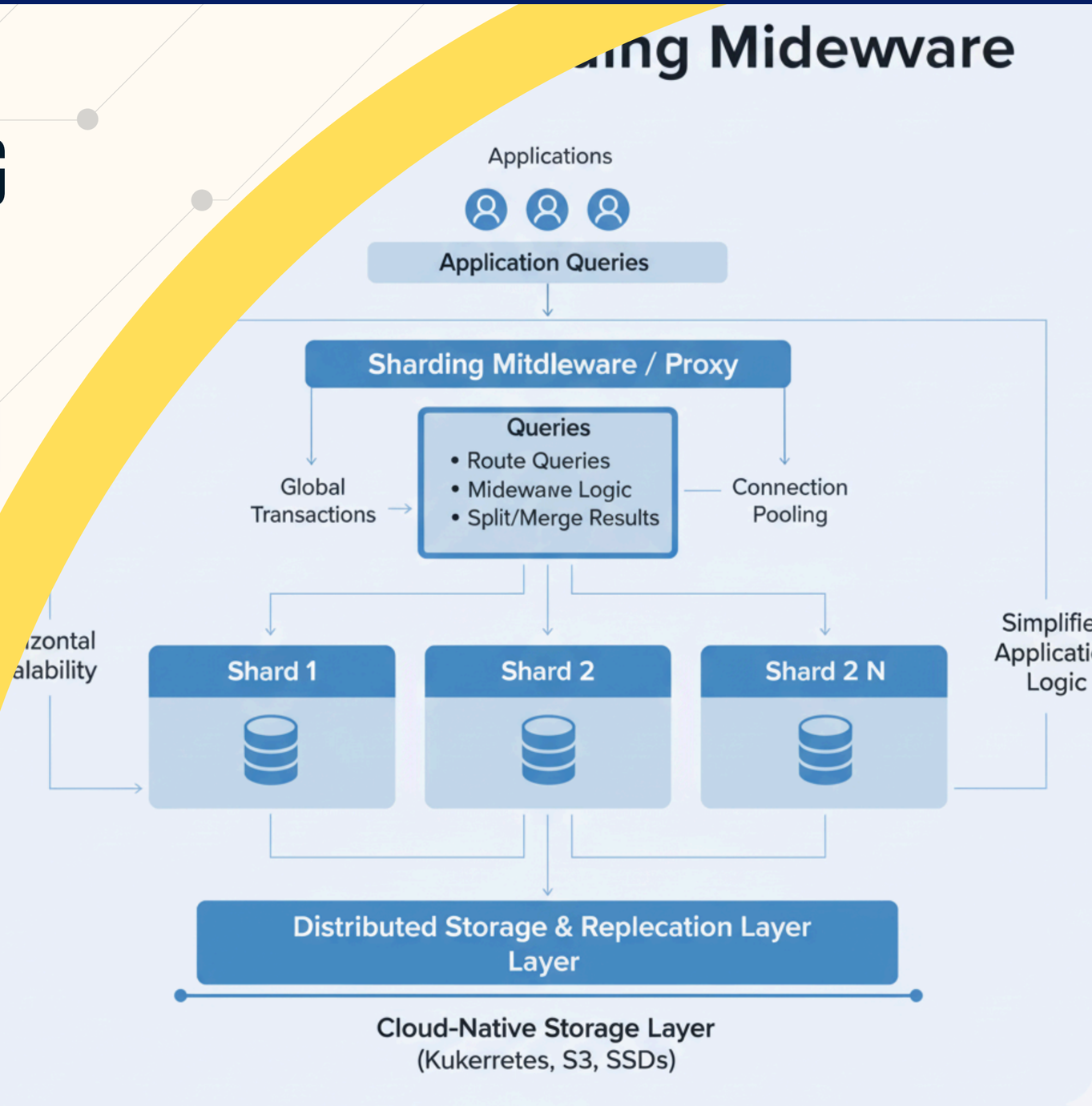
คือ การพัฒนา "Storage Engine" ใหม่ให้กับ MySQLเพื่อเพิ่มความสามารถในการ ประมวลผลเร็วขึ้น หรือ รองรับการกระจายข้อมูลบางส่วนโดยยังคงใช้ "MySQL Front-end" เดิม (เช่น syntax, client, tool) ได้ตามปกติ

- MySQL สามารถเปลี่ยน engine ได้ เช่น InnoDB, MyISAM
- นักพัฒนาจึงสร้าง engine ใหม่ ๆ ที่ปรับปรุงเรื่อง ความเร็ว / ขยายระบบ / In-memory processing

# TRANSPARENT SHARDING MIDDLEWARE

คือ "ซอฟต์แวร์ชั้นกลาง" ที่ช่วย แบ่งข้อมูล (shard) ออกหลายฐานข้อมูลโดยที่แอปพลิเคชัน "ไม่ต้องรู้เลย ว่ามีหลายเครื่อง เรียกว่า "transparent" เพราะ ทุกอย่างยังดูเหมือนเป็นฐานข้อมูลเดียว ต่อผู้ใช้งาน

- แอปพลิเคชันยัง **CONNECT** ผ่าน **MYSQL** เดิม
- **MIDDLEWARE** จะคอย จัดเส้นทาง **(ROUTING)** ของ **QUERY** ไปยัง **SHARD** ที่ถูกต้อง
- สามารถรวมผลลัพธ์กลับมาให้เหมือน **QUERY** เดียว



# ตัวอย่างฐานข้อมูล NEWSQL

| ฐานข้อมูล | จุดเด่นหลัก | ประเภทการใช้งาน | Cloud / Self-host |
|---|---|---|---|
| VoltDB | In-memory, ultra-fast transaction | Real-time system | Self-host / Cloud |
| NuoDB | Elastic, peer-to-peer, cloud-native | Enterprise, hybrid cloud | Cloud-first |
| MemSQL (SingleStore) | Hybrid (OLTP+OLAP), in-memory | Analytics + Transaction | Cloud / Hybrid |
| Google Spanner | Global ACID, TrueTime | Global-scale apps | Cloud-only (GCP) |
| CockroachDB | Open source, Raft consensus | Cloud-native distributed apps | Cloud / Self-host |

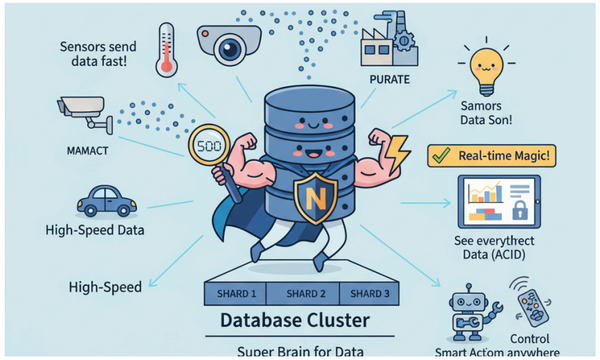
VOLTDB NUODB memSQL CockroachDB

# จุดเด่นของ NEWSQL

| หมวด | รายละเอียด |
|---|---|
| ความเร็วสูง | ใช้สถาปัตยกรรมใหม่ เช่น in-memory processing และ distributed execution ทำให้ประมวลผลได้รวดเร็วกว่า RDBMS เดิม |
| รองรับ ACID | คงคุณสมบัติความถูกต้องของข้อมูล (Atomicity, Consistency, Isolation, Durability) เหมือนฐานข้อมูลเชิงสัมพันธ์ |
| Scale-out ได้จริง | รองรับการกระจายข้อมูลหลายเครื่อง (horizontal scaling) ได้โดยไม่สูญเสียความสอดคล้องของข้อมูล |
| ใช้ SQL ที่คุ้นเคย | ใช้ภาษา SQL แบบเดิม ทำให้นักพัฒนาและ DBA สามารถใช้งานได้ทันทีโดยไม่ต้องเรียนรู้ใหม่ |

# ข้อจำกัดของ NEWSQL

| หมวด | รายละเอียด |
| --- | --- |
| การติดตั้งซับซ้อน | ต้องมีการตั้งค่าระบบ distributed, replication และ consensus protocols ที่ซับซ้อน |
| ยังไม่แพร่หลายเท่า NoSQL | ระบบและชุมชนผู้ใช้ยังมีน้อยเมื่อเทียบกับ NoSQL ที่ใช้กันอย่างกว้างขวาง |
| เหมาะกับ workload แบบ transaction | มีประสิทธิภาพสูงกับงานที่เน้นการทำธุรกรรม (OLTP) แต่ไม่เหมาะกับงานวิเคราะห์ข้อมูล (OLAP) |



# ตัวอย่างระบบที่ใช้ NEWSQL

ระบบธนาคาร, eCommerce, IoT ที่ต้องการความถูกต้องแบบ real-time

# แนวโน้มอนาคตของ NEWSQL

- Hybrid Database (SQL + NoSQL)
- ผสานกับ AI และ Analytics
- Edge Database สำหรับ IoT